PYTHON ACTIVITY RECORD

Student Email and Identifier Validation

1. Problem Statement

Develop a Python program that accepts a student's email address and registration ID, checks both values using simple validation rules, and displays either validation errors or a message confirming that both fields are valid.

2. Objectives

- Accept a student email address and registration ID from the user.
- Remove leading and trailing spaces from both inputs.
- Convert the registration ID to uppercase.
- Check that the email has no spaces, contains exactly one @ symbol, and has a dot in its domain.
- Check that the registration ID starts with CSE and has exactly seven characters with four digits after CSE.
- Display all detected validation errors or confirm both fields are valid.

3. Assumptions

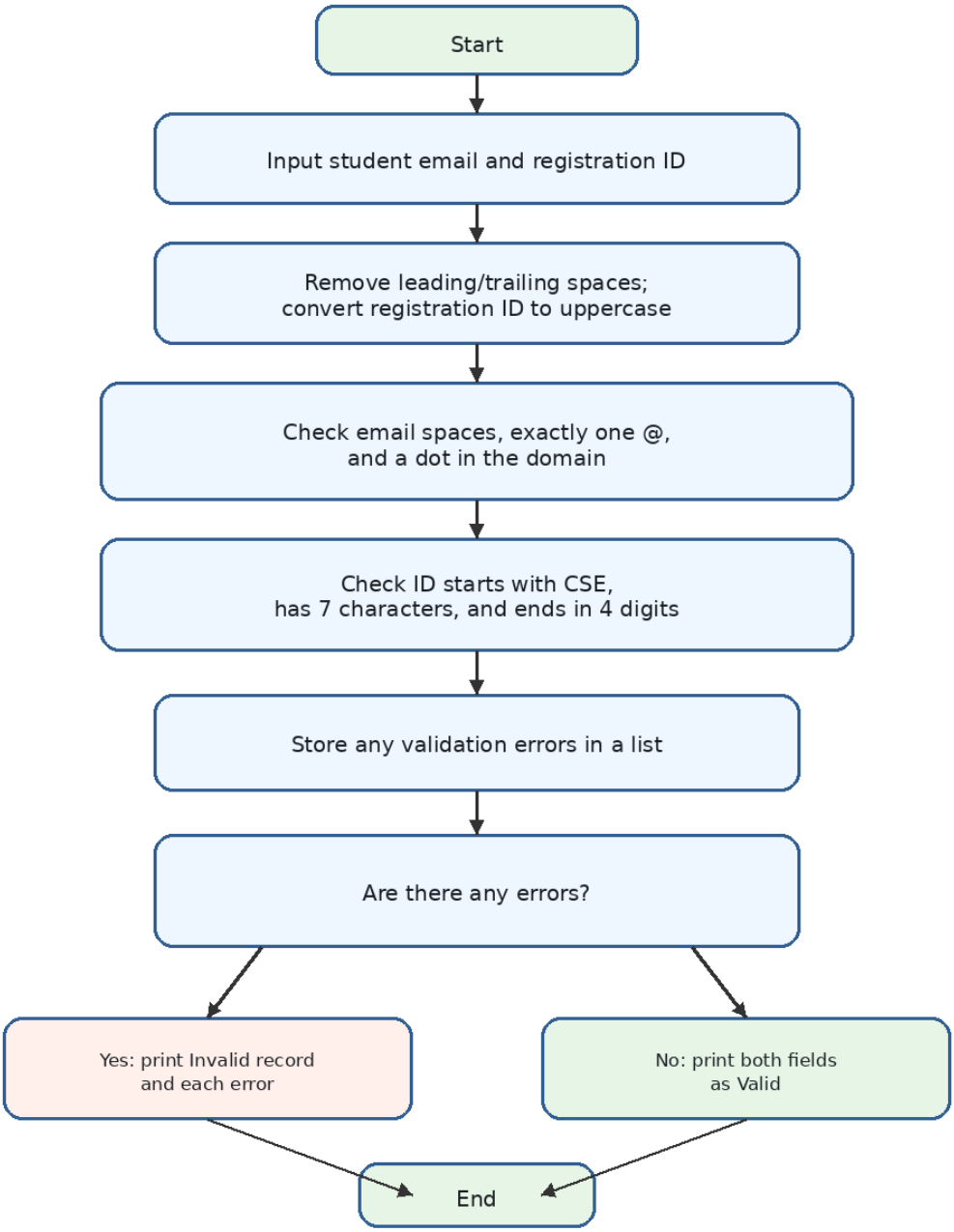
- A valid registration ID follows the format CSE1234: the prefix CSE followed by four digits.
- Email checking is basic format checking and does not confirm that the address exists or can receive mail.
- The program checks for spaces, exactly one @ symbol, and a dot in the domain; it is not a complete email-standard validator.
- Leading and trailing spaces are removed before validation.
- Registration IDs are case-insensitive because the input is converted to uppercase.

4. Algorithm

1. Start.
2. Read the student email and remove leading/trailing spaces.
3. Read the registration ID, remove leading/trailing spaces, and convert it to uppercase.
4. Create an empty list called errors.
5. If the email contains a space, add an email-space error.
6. If the email does not contain exactly one @, add an error; otherwise, if the domain part has no dot, add a domain error.
7. If the registration ID does not start with CSE, is not seven characters long, or does not end with four digits, add an ID error.
8. If errors exist, print Invalid record and list each error.
9. Otherwise, print that the email format and registration ID are valid.
10. Stop.

5. Flowchart

The flowchart shows the order of input, validation checks, and final output.



6. Pseudocode

```
BEGIN
    INPUT email
    REMOVE leading and trailing spaces from email

    INPUT reg_id
    REMOVE leading and trailing spaces from reg_id
    CONVERT reg_id to uppercase

    errors = empty list

    IF email contains a space
        ADD email-space error
    END IF
```

```
    IF email has not exactly one @
        ADD @-symbol error
    ELSE IF email domain has no dot
        ADD domain-dot error
    END IF

    IF reg_id does not start with "CSE"
       OR length of reg_id is not 7
       OR the last four characters are not digits
        ADD registration-ID error
    END IF

    IF errors is not empty
        DISPLAY "Invalid record:"
        DISPLAY each error
    ELSE
        DISPLAY "Email format: Valid"
        DISPLAY "Registration ID: Valid"
    END IF
END
```

7. Python Source Code

```
email = input("Student email: ").strip()

reg_id = input("Registration ID (CSE1234): ").strip().upper()

errors = []

if " " in email:

    errors.append("Email must not contain spaces.")

if email.count("@") != 1:

    errors.append("Email must contain exactly one @.")

elif "." not in email.split("@")[1]:

    errors.append("Email domain must contain a dot.")

if not reg_id.startswith("CSE") or len(reg_id) != 7 or not reg_id[3:].isdigit():

    errors.append("ID must start with CSE and end in four digits.")

if errors:

    print("Invalid record:")

    for error in errors:

        print("-", error)

else:

    print("Email format: Valid")

    print("Registration ID: Valid")
```

8. Test Cases

Test	Email input	Registration ID	Expected result
1	student@example.com	CSE1234	Both email format and ID are valid.
2	student example.com	CSE1234	Invalid record: Email must not contain spaces.
3	student@example.com	ABC12X4	Invalid record: ID must start with CSE and end in four digits.
4	student@@example.com	CSE1234	Invalid record: Email must contain exactly one @.
5	student@examplecom	CSE1234	Invalid record: Email domain must contain a dot.

9. Output Screenshots

Test Case 1: Valid email and ID

```
Python Output - Valid email and registration ID
Student email: student@example.com
Registration ID (CSE1234): CSE1234
Email format: Valid
Registration ID: Valid
```

Test Case 2: Invalid email

```
Python Output - Invalid email
Student email: student example.com
Registration ID (CSE1234): CSE1234
Invalid record:
- Email must not contain spaces.
```

Test Case 3: Invalid registration ID

```
Python Output - Invalid registration ID
Student email: student@example.com
Registration ID (CSE1234): ABC12X4
Invalid record:
- ID must start with CSE and end in four digits.
```

10. Conclusion

This program demonstrates input handling, string methods, conditional statements, lists, and validation in Python. It checks a student's email using simple rules and verifies that the registration ID follows the CSE plus four-digit format. It reports detected errors clearly. The email checks are basic and do not prove that an email address exists.